Paper Review • arXiv 2611.08449 • First submitted 2026-11-05

VFlash

Flash Primitive Decoding for Fast, Accurate Visual Planning

VFlash: Fast, Accurate Visual Planning via Flash Primitive Decoding

Yu-Cheng Chou, Shihao Wang, Jin Huang, Chi-Pin Huang, … , Alan Yuille, Linxi “Jim” Fan, Jan Kautz, Zhiding Yu

TL;DR

VFlash first declares a visual primitive's type and length via a tool call, then a lightweight drafter generates the whole box / trajectory / point motion in one forward pass, accepted if structurally valid, without token-level agreement with the AR model. Decoding is 12–42× faster, 3D trajectories are more accurate than AR, and downstream WAM success rises from 62.6% to 70.2%.

12–42×

Decoding speedup
Thor / Spark / H100， batch = 1

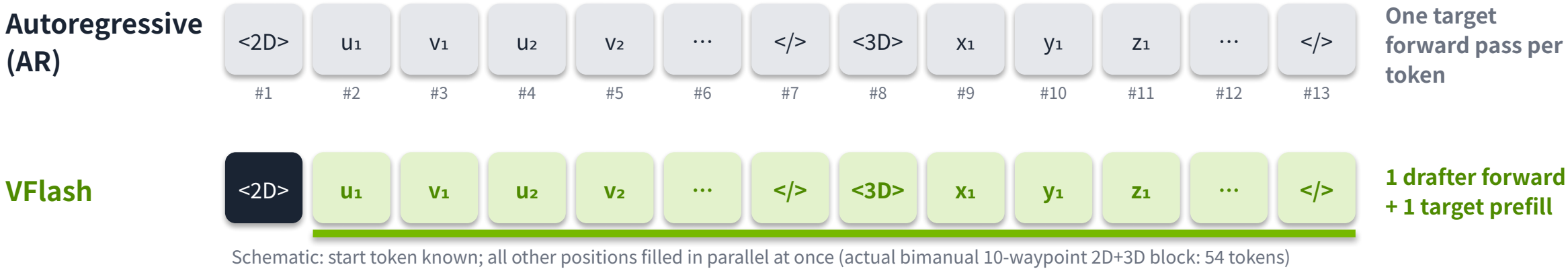
−28% ~ −61%

3D trajectory ADE reduction vs. AR
(2D+3D target, three datasets)

70.2%

MolmoSpaces avg. success rate
Baseline WAM: 62.6%

Problem: token-by-token AR coordinate generation is the latency bottleneck of embodied VLM visual planning



Schematic: start token known; all other positions filled in parallel at once (actual bimanual 10-waypoint 2D+3D block: 54 tokens)

54 / 84
tokens: 2-arm 2D+3D traj / Waymo driving

35–39 ms
per token on Thor: one 4B VLM forward pass

75–87%
AR decoding share on Thor of per-step end-to-end latency

2.2
DFlash in trajectory blocks: only tokens accepted per step

Speculative decoding (DFlash, etc.)

Requires token-level agreement with the target's greedy output; a 1-bin coordinate offset is rejected

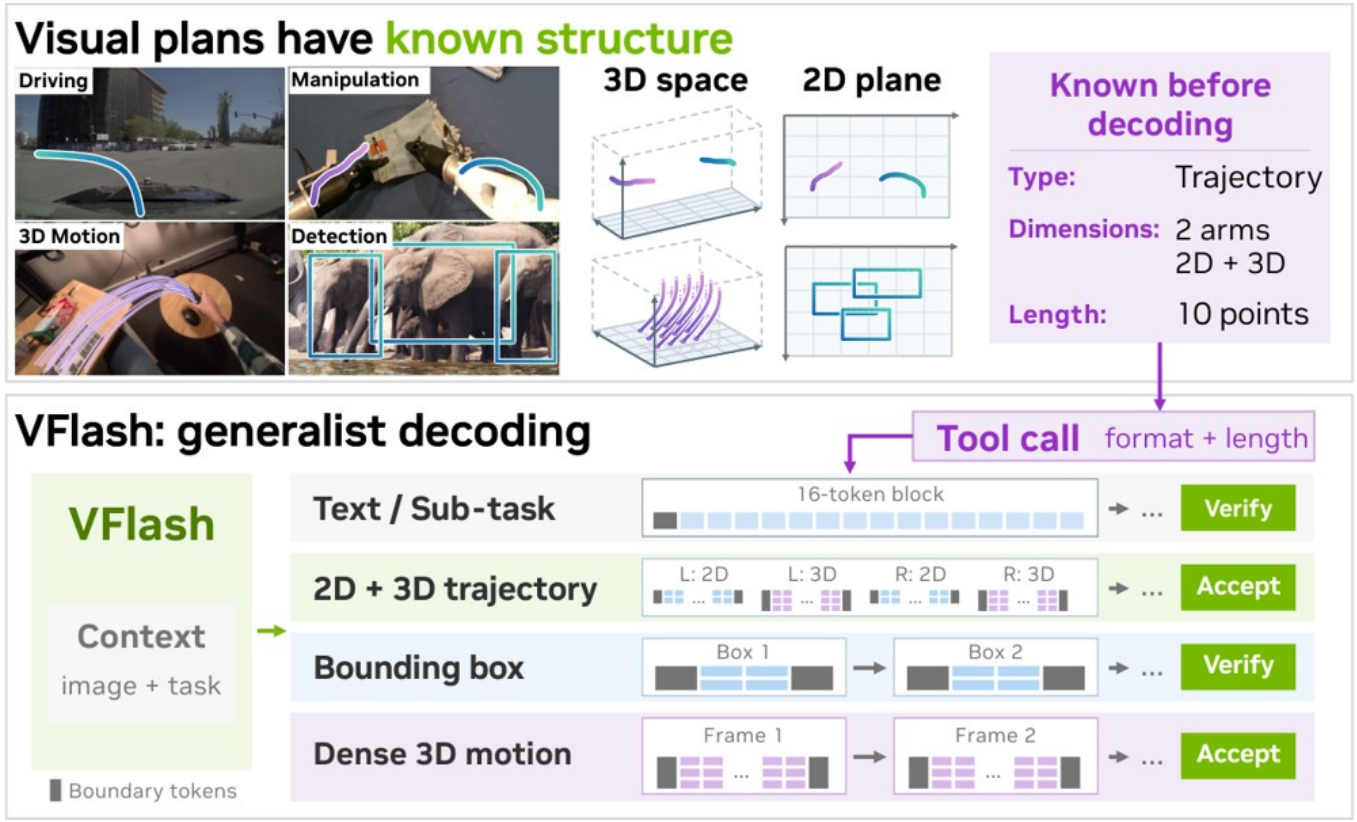
Diffusion LLMs

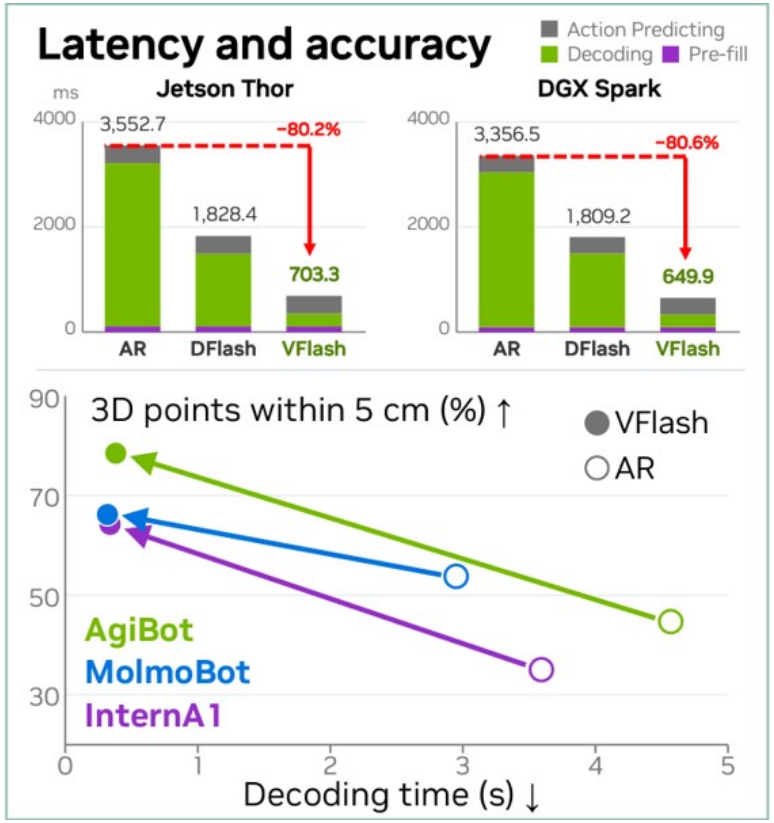
Parallel denoising, but often with a speed–quality trade-off

LocateAnything

Parallel decoding of boxes / points, but limited to fixed-size 2D primitives

Key insight: primitive structure is known before decoding, and spatial accuracy need not replicate AR

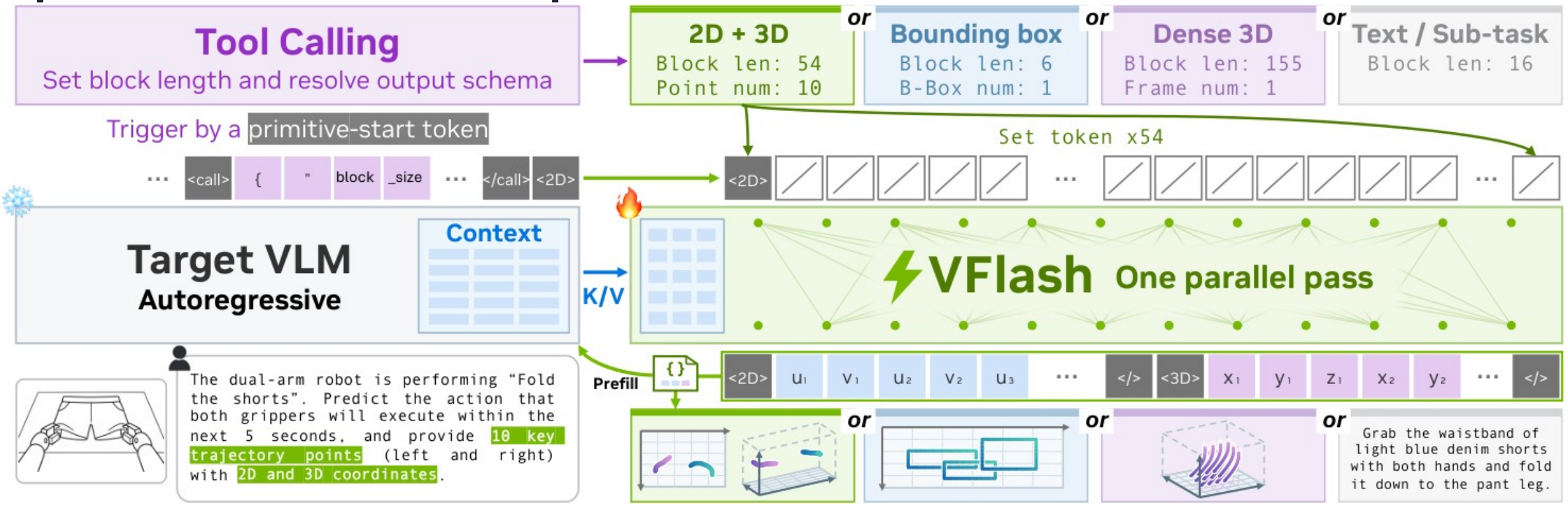




Known structure → whole-block parallelism
Once type + dimension + length are set, every slot's role is fixed, so the whole primitive can be predicted in one forward pass

Accuracy ≠ token agreement → direct acceptance
Accept the whole block whenever it is structurally valid; training also uses annotated coordinates as labels, not target-model continuations

FPD: a tool call declares the layout; the drafter generates the full primitive in one forward pass



① Tool call

Target VLM writes block_size as text, e.g., traj. 54 / box 6 / dense 155 / text 16

② Layout fixed

primitive-start token + length → each slot is a waypoint, axis or separator

③ One parallel pass

5-layer drafter uses frozen VLM layer 1/9/17/25/33 features as K/V to fill B(s)−1 masks

④ Direct accept

Commit whole block if schema-valid; one target VLM prefill updates KV cache and action-model features

Three design points: fixed-width layout, label-based training, accept if valid

Token representation

- Boundary-aligned: one block = one full primitive
- Position-aligned: each slot's role is predefined
- Special coordinate tokens <0>…<1000>; 1 coordinate = 1 position
- Textual numeric targets: sign + 5-digit fixed-width grid, rendered back to the original format
- Block length: box 6 | single-arm 2D+3D 5P+4 | bimanual 10P+8 | driving 2D+3D 4P+4 | dense 19P+3

Training

- AR target fine-tuned first, then frozen
- Drafter: 5 Qwen3 layers, 537M trainable params, shares the target's embedding and output layers
- Labels = annotated coordinates, not target-model continuations
- Equal weights within primitives; text positions keep DFlash decay weights
- One drafter serves both the text and primitive paths

Inference & acceptance

- Text: 16-token DFlash block + longest-prefix verification
- Primitives: per-position valid vocabulary plus cross-position rules (<oof> pairing, box-corner order), no extra forward passes
- Boxes: extra LocateAnything spatial-confidence guard
- On failure: roll back to the anchor; constrained target AR regenerates

$\tau_{dir} = B(s)$ if $A_s(\hat{a}) = 1$ (structurally valid)

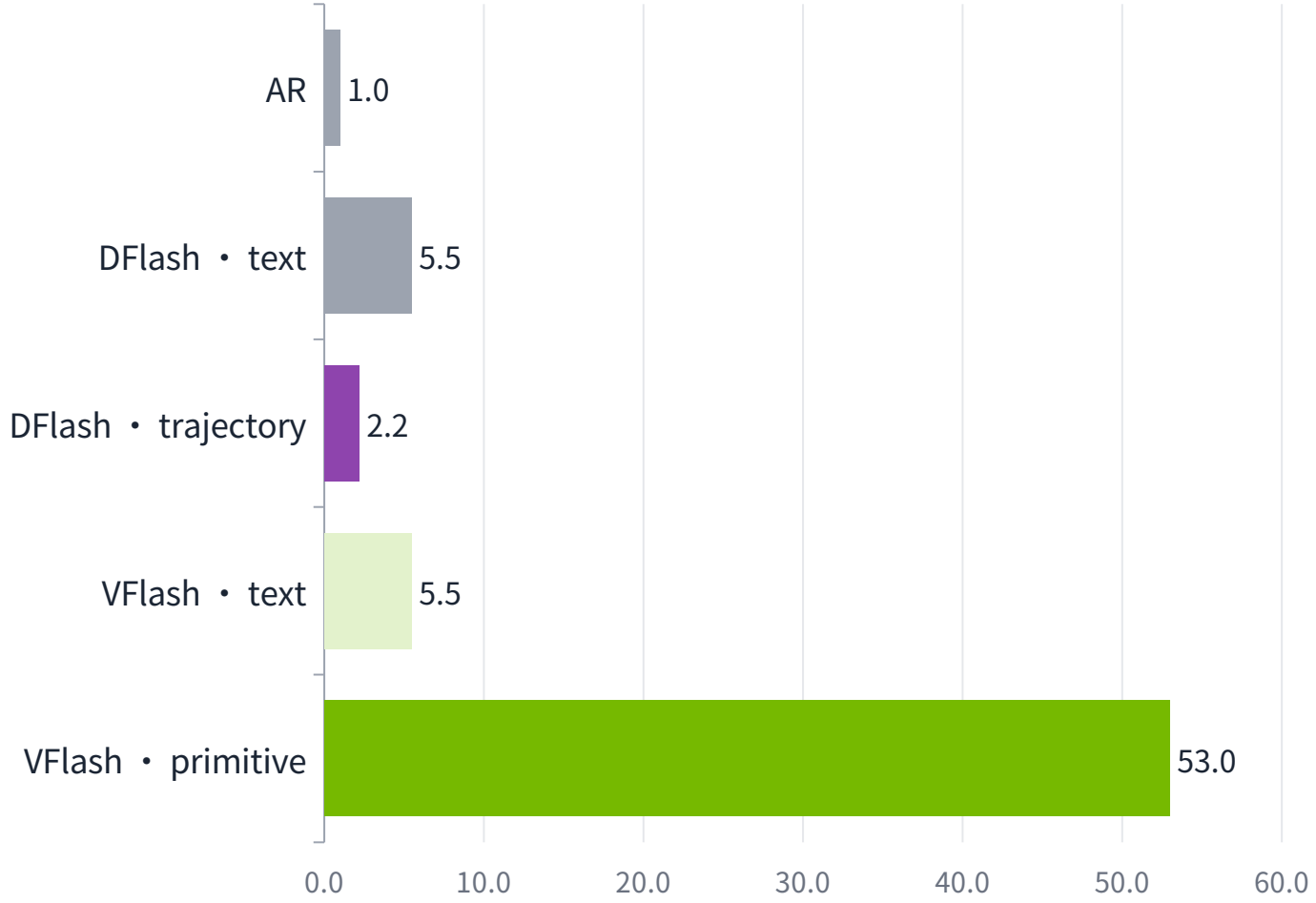
Otherwise $\tau = 1 + r$: keep the first r slots (r = 0 for trajectories); constrained AR regenerates the rest

$L_{dir} = (t_{par} + t_{target}) / B(s)$

Only 2 forward passes per primitive: finer plans enlarge B(s) but keep serial steps constant

Token-level verification slows trajectories: DFlash accepts only 2.2 tokens per step in trajectory blocks

Tokens committed per iteration (MolmoSpaces)



MolmoSpaces decoding time on Thor

3107.6 ms

AR: 88.75 target forward passes

1383.3 ms

DFlash: 29.5 iterations, ~halved

258.2 ms

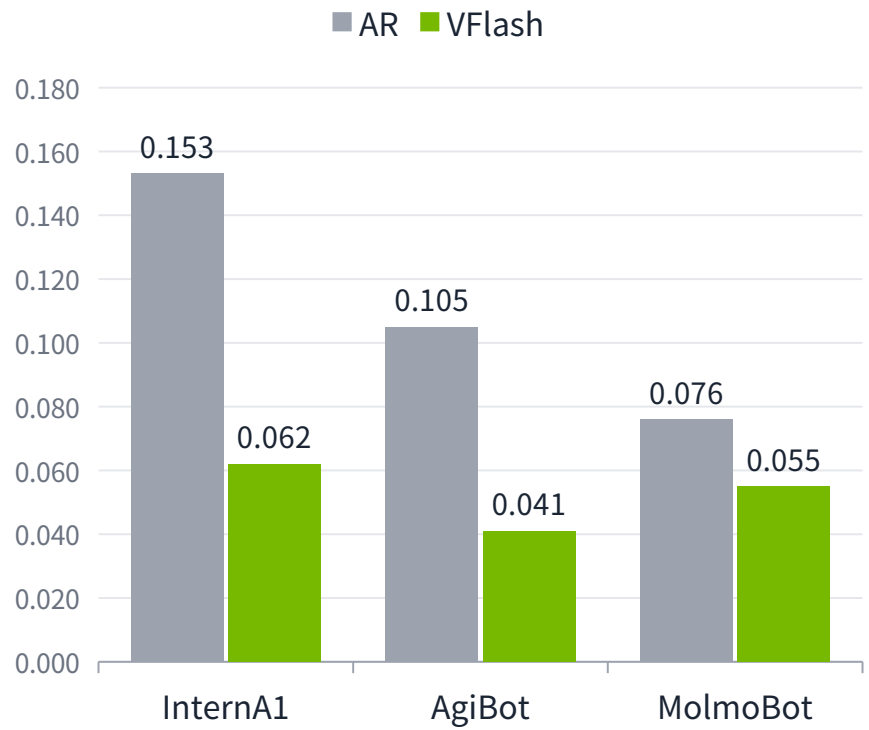
VFlash: 3.25 text + 1 primitive

Takeaway: a 1-bin coordinate offset is rejected by Eq. 2, yet the geometric error is tiny. Token agreement ≠ spatial accuracy.

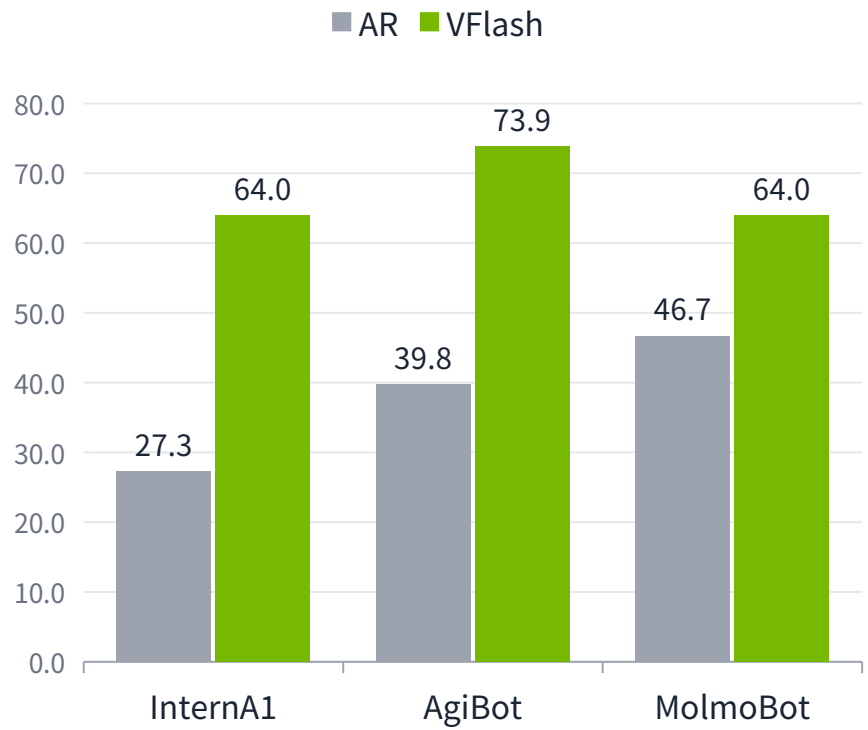
Source: paper § 3.3, App. E.2 decoding stats & per-stage latency table (MolmoSpaces)

3D robot trajectories: the one-pass drafter beats its own AR target

ADE (m, ↓)



SR@5cm （%，↑）



−28~61%

ADE reduction, 2D+3D target

−18~49%

ADE reduction, 3D-only target

−3~12%

Plus wrist-camera training (eval. still main camera only)

Observation: when AR outputs 2D in addition to 3D, its 3D ADE is 15–51% higher than 3D-only; VFlash generates 2D+3D in parallel with nearly unchanged accuracy. Across 3 training variants × 3 datasets × 4 metrics, VFlash beats AR everywhere.

Source: paper Table 4 (2D+3D training variant)

2D trajectories: small, consistent gains; boxes need AR fallback to retain accuracy

2D trajectory error (↓, distance / 1000)

Method	SR DFD	SR HD	SR RMSE	VA DFD	VA HD	VA RMSE
RoboBrain-2.0 7B	**0.167**	**0.158**	**0.125**	0.329	0.260	0.224
LoHo-Manip 4B	0.231	0.206	0.156	0.212	0.182	0.147
Baseline AR 4B	0.202	0.175	0.142	0.130	0.125	0.093
VFlash 4B	**0.194**	**0.162**	**0.133**	**0.126**	**0.116**	**0.089**

SR = ShareRobot-Bench, VA = VABench-V; green = best in column

Box F1 (%, ↑) and boxes per second, BPS (H100)

Method	Dataset	Mean	@0.50	@0.95	BPS
AR	COCO	**48.68**	65.29	**16.09**	3.15
AR	Dense200	60.10	76.70	**16.95**	6.73
Direct accept	COCO	41.32	59.64	10.76	5.96
Direct accept	Dense200	39.09	55.06	6.59	**22.66**
AR fallback	COCO	47.79	**66.38**	13.48	8.76
AR fallback	Dense200	**61.78**	**82.64**	12.11	17.46

- All 6 errors below the AR target (HD −7%, DFD / RMSE −3~6%)
- VABench-V: ~40% lower than best prior method LoHo-Manip
- ShareRobot: second only to RoboBrain-2.0

- With AR fallback: mean F1 on par with AR, 2.6–2.8× faster
- Higher F1@0.50 (Dense200 82.64 vs 76.70), lower F1@0.95
- No fallback: Dense200 mean F1 drops ~20 pts, but highest BPS

Dense 3D point motion: ~25× faster per rollout, mixed accuracy

≈25×

Per-rollout generation speedup
(preliminary, H100)

Dataset	AR (s)	VFlash (s)	Speedup
HOT3D	79.8	3.21	**24.8×**
WorldTrack	90.6	3.86	**23.5×**
DAVIS	99.6	3.78	**26.4×**

- Target: frozen MolmoMotion-AR; only the drafter is trained
- 8 points per frame = one 19P+3 fixed-width block; frames remain serial

PointMotionBench: VFlash vs. target MolmoMotion-AR

Dataset	Method	ADE ↓	FDE ↓	PWT ↑
HOT3D	MolmoMotion-AR	0.109	0.217	0.444
HOT3D	VFlash	0.122	**0.200**	0.391
WorldTrack	MolmoMotion-AR	0.143	0.261	0.445
WorldTrack	VFlash	0.145	0.300	0.408
DAVIS	MolmoMotion-AR	1.227	2.108	0.153
DAVIS	VFlash	**0.893**	**1.716**	0.143

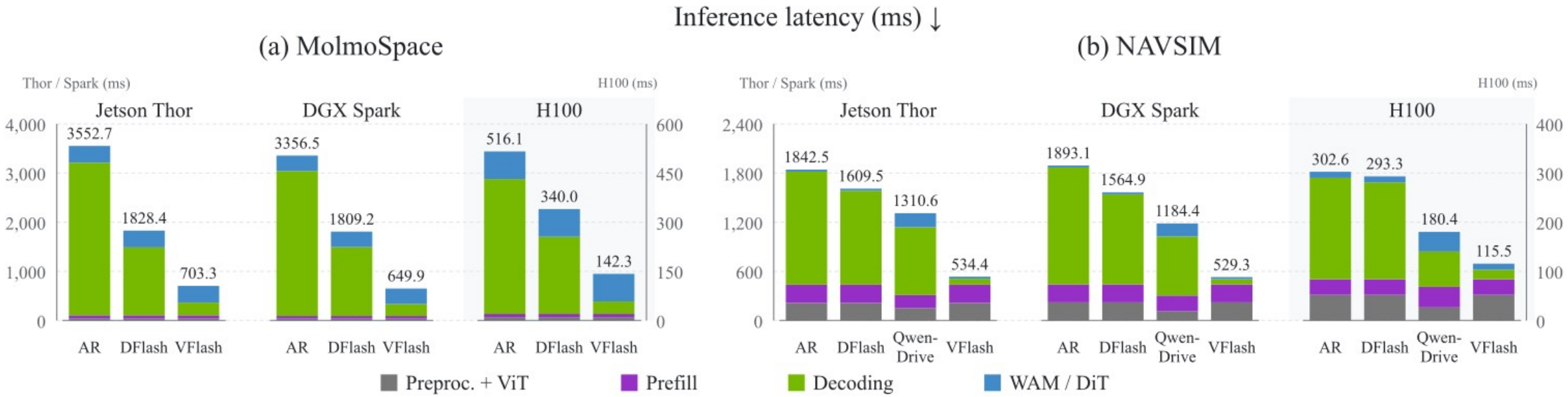
VFlash rows: green = better than AR, red = worse; ADE/FDE in meters

Better
DAVIS ADE −27% 、 FDE −19% ； HOT3D FDE −8%

Worse
WorldTrack FDE +15%; PWT −7~12% on all three datasets

Source: paper Table 5, App. F.3 (H100, batch = 1)

On-device latency: decoding nearly vanishes; MolmoSpaces ~5× faster end-to-end



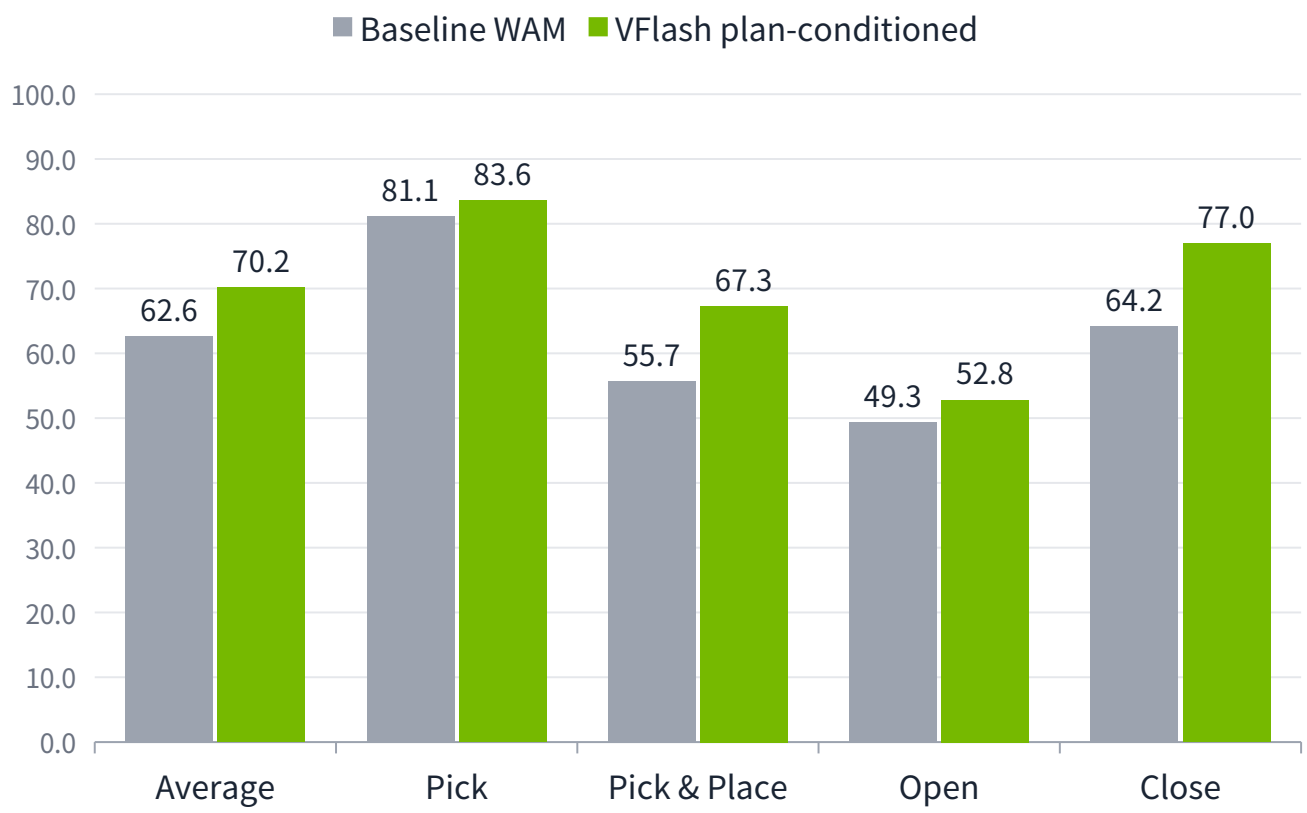
3552.7 → 703.3 MolmoSpaces · Thor （ms）	**≈5.1×**	
3356.5 → 649.9 MolmoSpaces · Spark （ms）	**≈5.2×**	
1842.5 → 534.4 NAVSIM · Thor （ms）	**≈3.4×**	
617.3 → 136.6 Waymo · H100 （ms）	**≈4.5×**	

Decoding 12–42× shorter, other stages unchanged; driving primitives grow from 36 to 84 tokens, yet Thor decoding only rises from 72 ms to 78 ms (AR: 35–39 ms per token forward).

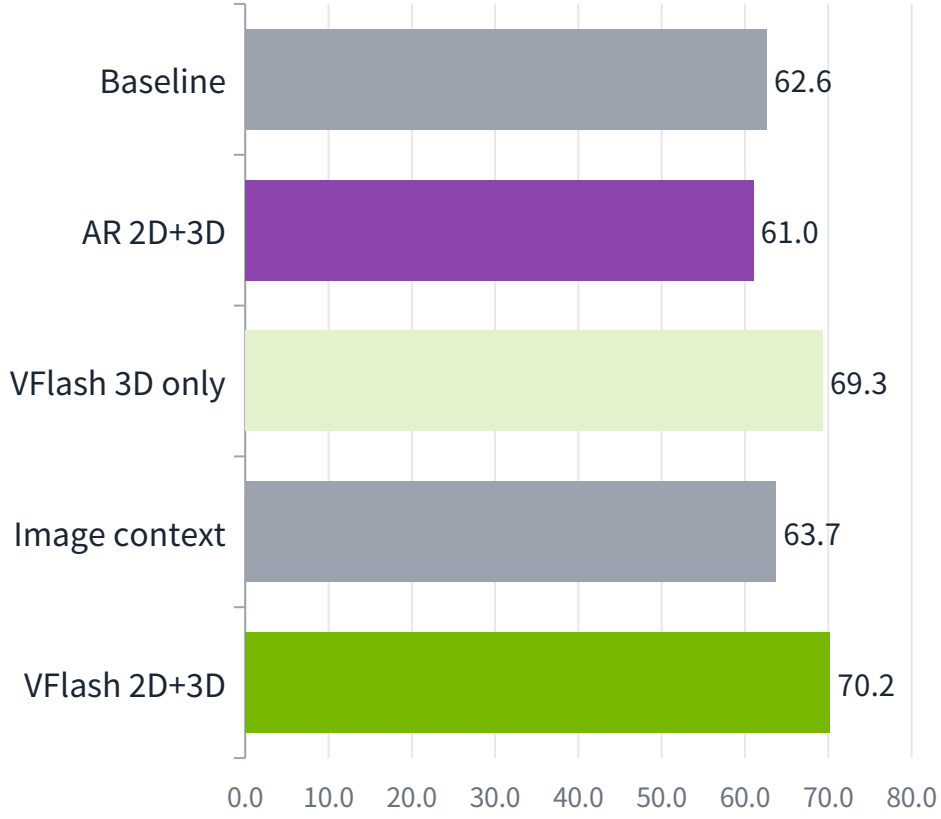
Source: paper Figure 4, App. E.2 per-stage latency table

MolmoSpaces: VFlash plans lift WAM avg. success from 62.6% to 70.2%

Per-task success rate (%)



Ablation: plan representation & context (avg. %)



+7.6
Avg. success-rate gain (pts)

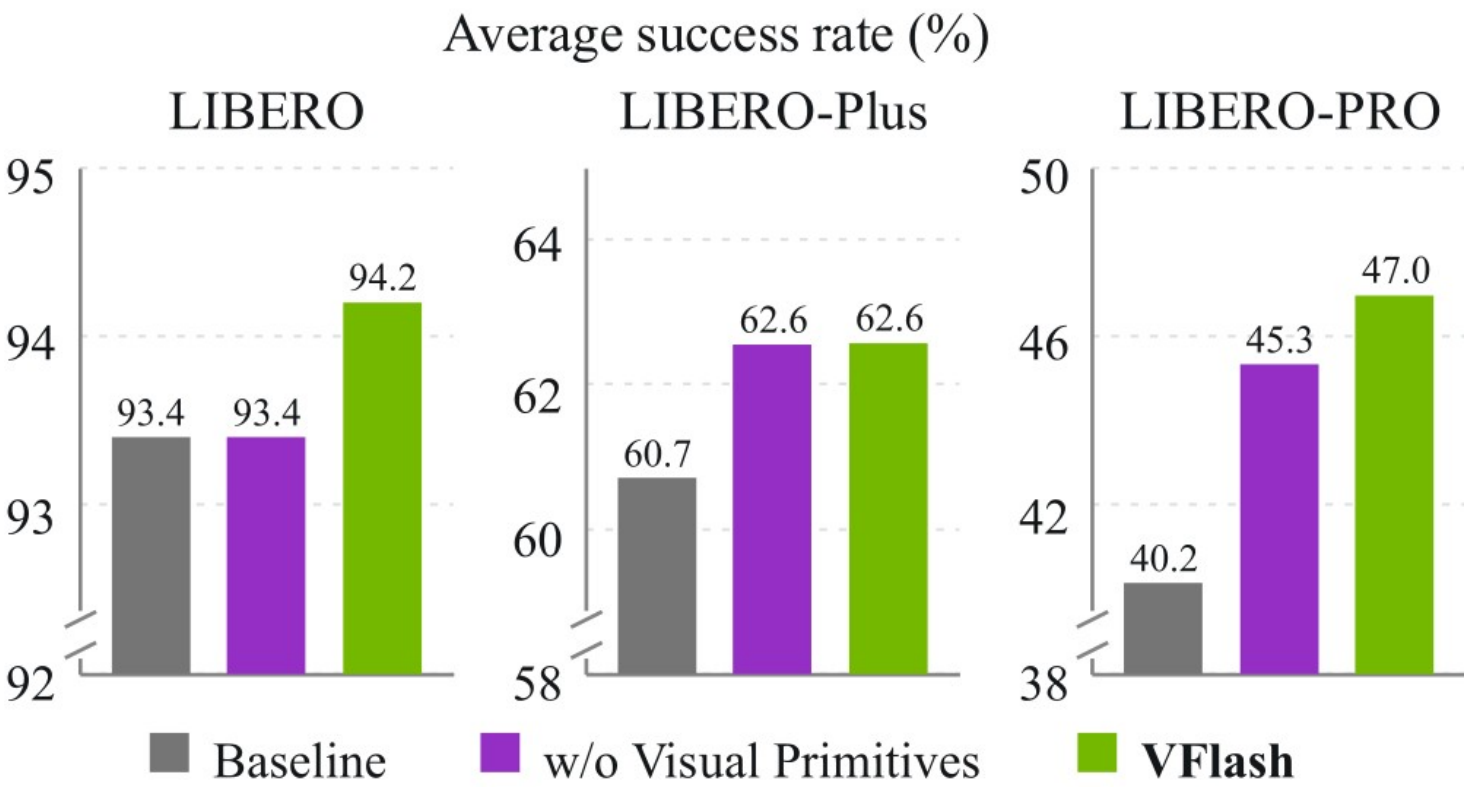
+11.6 / +12.8
Pick&Place / Close gains

+9.9
Above PrimeR0 (60.3%)

61.0%
AR plans: below baseline

Source: paper MolmoSpaces main & ablation tables (§ 3.2)

LIBERO suites: the stronger the perturbation, the larger the gain from visual plans



Avg. success-rate gain over baseline

+0.8
LIBERO （93.4 → 94.2）

+1.9
LIBERO-Plus （60.7 → 62.6）

+6.8
LIBERO-PRO （40.2 → 47.0）

- LIBERO-PRO: Spatial / Object / Goal +11.1 / +9.4 / +6.9 over baseline
- Vs. "subtasks only": Spatial +2.6, Goal +1.2, Long +2.0; trajectories add info beyond subtasks
- All policies trained only on the standard 40 LIBERO tasks

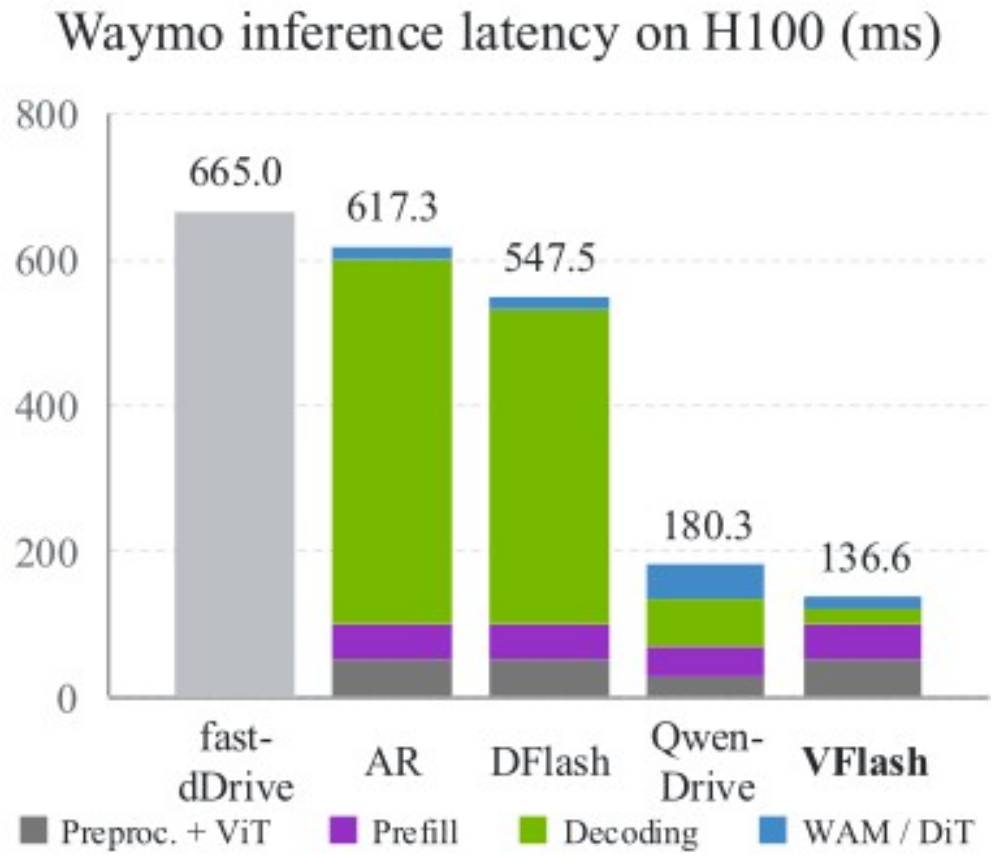
Source: paper Figure 3, § 3.2

Autonomous driving: frozen VLM + small action head, competitive scores, lower latency

NAVSIM PDMS and Waymo RFS (↑)

Method	RL	NAVSIM PDMS	Waymo RFS
ReCogDrive	–	86.8	–
ReCogDrive	✓	89.6	–
Qwen-Drive-1.0-SFT	–	88.2	7.78
Qwen-Drive-1.0-RL	✓	**90.7**	**7.91**
VFlash-SFT	–	**87.1**	**7.79**
VFlash-RL	✓	**90.7**	**7.86**

- SFT already beats ReCogDrive (87.1 vs 86.8); RL tunes only the action head, plans unchanged
- NAVSIM 90.7 ties Qwen-Drive-1.0-RL; Waymo 0.05 below the best (7.91)
- Caveat: Waymo RL checkpoint selected on 479 validation scenarios (in-sample)



Waymo • H100： 136.6 ms vs AR 617.3 / Qwen-Drive 180.3

Fast-dDrive 665 ms is its paper-reported value, not measured in the same setup

Limitations and caveats

Accuracy not uniformly better

- Boxes: F1@0.95 drops (COCO 16.09 → 13.48)
- No fallback: Dense200 mean F1 60.10 → 39.09
- Dense motion: HOT3D / WorldTrack ADE and PWT on all three datasets trail AR

Asymmetric comparison (reviewer's view)

- AR target only gets next-token fine-tuning; the drafter trains ~3 more passes on the same data
- Paper does not separate how much of the accuracy gain comes from extra training vs. parallel prediction

Limited downstream statistical power

- MolmoSpaces / LIBERO: only 1 training seed + 1 eval seed per config
- Gain on standard LIBERO only +0.8
- Waymo RL checkpoint selected on the same 479 validation scenarios (in-sample)

Timing protocol

- Dense-motion 25× is a preliminary single-rollout measurement; frames remain serial
- MolmoSpaces AR decoding time from an earlier separate profile
- Tool-call JSON given directly during timing, not counted as generation

Source: paper § 3, App. B/D/E/F; item 2 is the reviewer's view

Summary: AR doesn't cap visual-output quality

01

Structure = parallelism

Primitive type, dimension and length are known before decoding; with blocks aligned to primitives, tens to hundreds of tokens are generated in one forward pass.

02

No token agreement: fast and accurate

Accept if structurally valid, train on annotated coordinates: 3D trajectory ADE −18–61%, decoding 12–42× faster.

03

System-1-style visual planning

~80% lower on-device latency; plan-conditioned WAM rises from 62.6% to 70.2% on MolmoSpaces and +6.8 on LIBERO-PRO.

In one sentence: VFlash uses "tool call declares structure + one forward pass per primitive + accept if structurally valid" to shift embodied VLM visual planning from per-token to per-primitive generation, making trajectories both faster and more accurate; accuracy trade-offs for boxes and dense motion, and single-seed downstream results, need further validation.